

MEMBER 1 — Platform & Backend Orchestration Lead (Bhavya)

This person controls how everything talks to each other.

If this layer fails, nothing works.

♦ **What This Member Actually Builds**

1 Authentication System

- GitHub OAuth flow
- Store GitHub access token securely
- Generate JWT
- Validate JWT for all requests

Why important?

Because:

- Private repo cloning depends on token
 - VS Code extension authentication depends on JWT
 - API security depends on it
-

2 Scan Request Handling

Endpoints like:

- POST /scan-repo

- POST /scan-url
- POST /scan-local

This person ensures:

All three routes eventually call the same scan logic.

That's architectural maturity.

3 Redis Job Queue

This person must understand:

Why scanning cannot block HTTP.

Flow:

Client → API → Redis → Worker

They implement:

- Job creation
 - Job status tracking
 - Job failure handling
-

4 Database Design

Tables:

Users

Projects

Scans

Vulnerabilities

Scan_Events

AI_Responses

This person defines relationships properly.

♦ **What This Member Must Learn**

- OAuth basics
 - JWT authentication
 - REST API design
 - Redis queue basics
 - PostgreSQL schema design
 - Error handling & logging
-

 MEMBER 2 — Security & Scan Engine Lead (Divyansh)

This is the most technically sensitive role.

This person handles untrusted code.

♦ **What This Member Actually Builds**

1 Docker Isolation

For each scan:

- Create container
- Mount project
- Run tools

- Destroy container

Must set:

- CPU limits
- Memory limits
- Timeout

This protects system.

2 Semgrep Integration

This person learns:

- How to run CLI
 - How to parse JSON output
 - What rules detect what vulnerability
-

3 Trivy Integration

This person learns:

- CVE detection
 - Dependency scanning
 - Container scanning
-

4 Normalization Layer

Very important.

Convert different outputs into one schema:

```
{  
  scan_id,  
  layer,  
  vulnerability_type,  
  severity,  
  cvss,  
  file,  
  line,  
  description  
}
```

Without normalization → chaos.

♦ What This Member Must Learn

- Docker basics
 - CLI tool execution
 - JSON parsing
 - OWASP basics
 - CVSS basics
-

MEMBER 3 — Risk & Heat Modeling + Web Dashboard Lead (Bhumika)

This is your intelligence layer.

This person transforms raw findings into insight.

◆ What This Member Actually Builds

1 Risk Score Formula

Takes:

- CVSS
- Severity
- Layer weight
- Persistence factor

Produces:

0–100 risk score

2 Heat Score Calculation

Compute:

Vulnerability heat

File-level heat

Project-level heat

This powers heat map.

3 React Dashboard

Build:

- File tree heat coloring
- Risk meter
- Trend graph

- Vulnerability filters
- Scan comparison

This is user-facing intelligence.

♦ What This Member Must Learn

- CVSS scoring logic
 - Basic risk modeling math
 - React visualization
 - Chart libraries
 - UI design for security tools
-

MEMBER 4 — AI & VS Code Extension Lead (Divyanshi)

This is your differentiation feature.

♦ What This Member Actually Builds

1 VS Code Extension

- Create extension project
- Add command “Scan Project”
- Zip project

- Send to backend
 - Receive vulnerabilities
-

2 Heat Visualization in IDE

- Highlight lines
 - Color intensity based on severity
 - File heat badges
-

3 AI Integration

When user clicks vulnerability:

- Send code snippet to backend
 - Backend sends to LLM
 - Return explanation
 - Display fix suggestion
-

♦ What This Member Must Learn

- VS Code API basics
- Editor decorations
- Webview panels

- Prompt engineering
 - API integration
-

◆ HOW THEY WORK TOGETHER

Important.

Member 2 → Member 3
(Security output → Risk engine)

Member 3 → Member 4
(Heat data format consistency)

Member 1 → All
(API contract enforcement)